A STUDENT'S GUIDE · AUSTRALIAN INFORMATICS OLYMPIAD

The Sliding Window & Two Pointers

Turning $O(N^2)$ scans into $O(N)$ sweeps



A self-study guide that takes you from the core idea to full-marks solutions. You'll work through two classic AIO problems — **Mansion** (2007) and **Wet Chairs** (2015) — learn the **two shapes** every two-pointer problem takes, and then code two exercises of your own, each with test data to check yourself against.

What you need first: comfort with C++ basics — arrays, `for` loops, `if` / `while`, and reading input with `cin`. All code here is plain C++ (fixed-size arrays, `iostream`) — nothing fancy.

How to use this guide

Read it with a compiler open. Every worked program is complete and correct — type it in, run it, and poke at it. When you reach a **CHECK YOURSELF** box, try to answer before reading on. When you reach the two exercises near the end, give each a real attempt *before* looking at the worked solutions in the appendix — the struggle is where the learning is.

The one sentence to remember
Keep a running summary of the current window, and **repair** it in O(1) as the window's edges move forward and never backward. That single trick is the whole technique.

WHAT'S INSIDE

1. The core idea — repair, don't recompute
2. Fixed-size window — *Mansion*
3. Variable-size window — the two pointers, monotonicity, and why it's linear
4. The two shapes of a two-pointer problem (the part most people get muddled)
5. Worked example — *Wet Chairs*
6. Your recipe, common pitfalls, and when *not* to use it
7. Exercises: *Assembly Line* and *Snack Bar*
8. Cheat sheet · Appendix: worked solutions & more problems

1 The core idea: repair, don't recompute

Lots of problems ask you to look at every **contiguous block** of an array — every stretch of K in a row, or every run that satisfies some rule — and compute something about it: a sum, a count, a minimum. The obvious way is to take each block and add it up from scratch. If there are about N blocks and each costs about N to measure, that's **O(N²)**, which is usually too slow for full marks once N reaches 10^5.

Here's the saving idea. Neighbouring blocks **overlap almost completely**. When a window slides one step to the right, only two things change: one element drops off the left, and one joins on the right. So don't rebuild — **mend**:




```
new_sum = old_sum  -  a[element that left]  +  a[element that entered]
```

That's one subtraction and one addition — **O(1)** per window instead of O(N). Do that at every step and the whole scan becomes **O(N)**. Everything else in this guide is a variation on two questions: *what summary do you keep*, and *how do the two edges move*.

2 Fixed-size window — *Mansion* (AIO 2007)

The problem, in one line. Given an array of N values, find the maximum sum of any K consecutive values.

The slow way. For each of the ~N starting positions, add up K values. That's O(NK) — fine for easy subtasks, too slow for the big ones.

The insight. Two neighbouring windows share all but their two edge cells. Moving from the window starting at 0 to the one starting at 1, you lose `a[0]` and gain `a[K]`; the middle is untouched. So keep a `window` total and slide it:

```cpp
#include <iostream>
using namespace std;

const int MAXN = 100005;
long long a[MAXN];

int main() {
    int N, K;
    cin >> N >> K;
    for (int i = 0; i < N; i++) cin >> a[i];

    long long window = 0;
    for (int i = 0; i < K; i++) window += a[i];   /* first window */
    long long best = window;
    for (int i = K; i < N; i++) {
        window += a[i] - a[i - K];                /* repair, don't recompute */
        if (window > best) best = window;
    }
    cout << best << "\n";
    return 0;
}
```

Mansion — complete and verified against a brute force on 2000 random cases.

Why it's O(N). You pay O(K) once to build the first window, then O(1) for each of the remaining N−K slides. Total: O(N). (The prefix-sum method gets the same complexity; sliding window just uses no extra array.)

Check yourself — why does the loop start at `i = K`, not `i = 0`?

Because the window for the first K elements is already built before the loop. The loop handles each *new* right edge from index K onward, and `a[i] - a[i-K]` is the entering-minus-leaving update for that step.

Check yourself — the values can be negative. Does the code still work?

Yes. Nothing assumes positivity; `best` starts at the first window's sum (not 0), so an all-negative array returns the least-negative window rather than a wrong 0.

3 Variable-size window — the two pointers

In a fixed window the width never changes, so both edges march together. The richer pattern lets the two edges move **independently**: one races ahead to make the window "good enough," the other follows to tighten it. Two indices chase each other down the array — and crucially, **neither ever moves backward**. That forward-only property has a name and a payoff.

Monotonicity — the property that makes it correct

Suppose you're looking for the shortest block that meets some target, and for left edge = 3 the earliest right edge that works is 7. Now push the left edge to 4. The earliest working right edge **can't drop below 7** — it can only stay or move further right. Why? Advancing the left edge *removed* an element, and removing an element can't turn a not-yet-good block into a good one. Whatever the right edge had to reach before is a floor for where it must reach now. So the right edge never needs to back up.

This is the whole justification

Whenever you can argue "as the left edge moves right, the required right edge only moves right," a two-pointer sweep is both **correct** and **fast**. If you can't make that argument, the technique may give wrong answers — not just slow ones. Always check it.

Why it's O(N), even though it looks nested

A two-pointer sweep usually reads as an outer loop with an inner `while` — which *looks* like $O(N^2)$. It isn't. Don't count loop nesting; **count pointer moves**. The left index advances at most N times over the whole run. The right index also advances at most N times over the whole run, because its position is *remembered* between outer steps and never reset. At most 2N moves, each O(1): **O(N)** total.

The name for this argument

It's called **amortised analysis**: one outer step might do a lot of inner-loop work, but that work is "charged" to the right pointer's forward progress — and the right pointer only has N units of progress to spend across the entire run.

4 The two shapes of a two-pointer problem

This is the part that trips people up. "Two pointers" isn't one template — it's two mirror-image templates, and they suit opposite

kinds of questions. Read the problem's wording to tell them apart.

Longest / at-most	Shortest / at-least
LONGEST THAT STAYS UNDER	SHORTEST THAT REACHES
Question sounds like: "the *longest* run whose total is *at most* B / that has *no more than* k of something."	**Question sounds like:** "the *shortest* block that has *at least* N chairs / reaches some threshold."
Move: add on the right every step; if that breaks the rule, **shrink from the left** until it holds again. The window is valid at every step, so you record a candidate each time.	**Move: grow the right** until the window qualifies, record, then drop the left element and move on. Adding helps you qualify, so the right edge leads.
Maps to: the *Snack Bar* exercise (longest run within a budget).	**Maps to:** *Wet Chairs* (shortest block meeting a requirement).

Here are the two skeletons side by side. Notice the roles swap: which index the `for` drives, whether the inner `while` *shrinks* or *grows*, and where "record the answer" sits.

Longest / at-most

```
int l = 0;
long long cur = 0;                  // running summary of the window [l, r]
for (int r = 0; r < n; r++) {
    cur += a[r];                    // element r ENTERS on the right
    while (cur > limit) {           // adding a[r] broke the rule
        cur -= a[l];                // element l LEAVES on the left
        l++;
    }
    best = max(best, r - l + 1);    // window is valid at every step
}
```

Shortest / at-least

```
int r = 0;                          // window is half-open [l, r)
long long cur = 0;
for (int l = 0; l < n; l++) {
    while (r < n && cur < target) {   // not enough yet
        cur += a[r];                  // grow on the right
        r++;
    }
    if (cur >= target)
        best = min(best, r - l);      // shortest qualifying window
    if (r > l) { cur -= a[l]; }       // l leaves; r stays put
    else { r = l + 1; }               // window was empty: keep r with l
}
```

The empty-window guard
In the shortest/at-least form, the left edge can catch up to the right and empty the window. That's why the skeleton has `if (r > l) ... else r = l + 1;` — without it, you'd "remove" an element that isn't there and your counts would drift. You'll see this exact guard in the *Wet Chairs* code next.

5 Worked example — *Wet Chairs* (AIO 2015) SHORTEST / AT-LEAST

The problem, in one line. There are C chairs in a row, each wet or dry. Seat N friends in one contiguous block; at most K of them tolerate a wet chair. So the block needs **at least N chairs** and **at least N−K dry ones**. Find the **shortest** such block.



1 = dry, 0 = wet. With C=10, N=5, K=2 you need ≥5 chairs and ≥3 dry; the shortest valid block here has length 5.

This is the shortest/at-least shape. Keep two running counts — how many chairs and how many dry ones are in the window — grow the right edge until the window qualifies, record its length, then advance the left edge. The right index `r` is declared *outside* the outer loop, so it never restarts: that one detail is the difference between O(C) and O(C²).

```cpp
#include <iostream>
using namespace std;

const int MAXC = 100005;
int dry[MAXC];   /* dry[i] = 1 if chair i is dry, else 0 */

int main() {
    int C, N, K;
    cin >> C >> N >> K;
    for (int i = 0; i < C; i++) cin >> dry[i];

    int need_dry = N - K;
    int best = C;
    int num_chairs = 0, num_dry = 0;
    int r = 0;                        /* window is half-open [l, r) */
    for (int l = 0; l < C; l++) {
        while (r < C && !(num_chairs >= N && num_dry >= need_dry)) {
            num_chairs++;
            num_dry += dry[r];
            r++;
        }
        if (num_chairs >= N && num_dry >= need_dry) {
            if (num_chairs < best) best = num_chairs;
        }
        if (r > l) { num_chairs--; num_dry -= dry[l]; }
        else { r = l + 1; }
    }
    cout << best << "\n";
    return 0;
}
```

Wet Chairs — complete; verified against a brute force on 16,000 random cases.

Check yourself — where would this silently become O(C²)?

If you moved `int r = 0;` inside the `for` loop. Then the right scan would restart from the current left edge every time, re-counting chairs you'd already seen. Keeping `r` outside is what makes each chair enter the window exactly once.

6 Your recipe, the pitfalls, and the limits

Three questions to ask before you code

1. **What summary do I keep?** A sum, a count, a maximum, a small frequency table — whatever the question measures about a window.
2. **How does the summary change when one element enters and one leaves?** Both updates must be O(1) (or close), or you lose the whole benefit.
3. **What is monotonic here?** Find the reason a pointer can move forward and never come back. If you can't, two pointers may not apply.

Common pitfalls

Recomputing instead of repairing

If you re-sum the whole window inside the loop, you're back to O(N²). Touch only the entering and leaving elements.

Resetting the fast pointer

In a two-pointer sweep, the right index must live *outside* the outer loop. Restart it and you quietly drop back to O(N²).

Off-by-one at the edges

Decide up front whether your window is inclusive `[l, r]` or half-open `[l, r)` and never mix them. Half-open (where `r` is "one past the last element") tends to have fewer ±1 bugs.

Forgetting to remove the leaver / empty windows

When the left index advances, subtract its element from every count. And when the left index catches the right, guard against an empty window so counts can't go negative.

When two pointers does not apply

The method needs that "required edge only moves one way" property. A classic trap: "longest run with sum ≤ B" works beautifully when all values are **positive** (shrinking the left only lowers the total), but **breaks with negative numbers** — removing a negative can *raise* the total, so the right edge might need to retreat. When monotonicity fails, reach for a different tool (prefix sums with binary search, a heap, or divide and conquer) instead of forcing the sweep.

7 Exercises — now you write the code

Two problems: one of each shape. Each ships with 10 test cases (in the class pack) covering the awkward edges — a single element, the whole array, all-zero input, an answer of 0, and a large N=100000 case that an O(N²) attempt will fail on. Try hard before turning to Appendix A.

Exercise 1 — Assembly Line FIXED WINDOW

A line has N stations; station i produced `d[i]` defects. You'll shut down a run of exactly K **consecutive** stations for maintenance and want to disturb quality least — so find the K consecutive stations with the **smallest** total defects. Report that total.

Input: line 1 has N and K ($1 \le K \le N \le 10^5$); line 2 has N integers d[i] ($0 \le d[i] \le 10^9$). **Output:** the minimum total. Totals overflow 32 bits — use `long long`.

```
Input
6 3
7 2 5 1 4 6
```

```
Output
8
```

The four blocks of 3 total 14, 8, 10, 11; the smallest is 8 (from 2 + 5 + 1).

> **Hint.** It's *Mansion* with `min` in place of `max`. Build the first K as a window, then `window += d[i] - d[i-K]` each step and keep the smallest. No inner loop.

Exercise 2 — Snack Bar LONGEST / AT-MOST

A bar lays out N items in a row; item i costs `p[i]`. With a budget B, buy the **longest** run of **consecutive** items whose total cost is at most B. Report how many items that is (0 if even one item is unaffordable).

Input: line 1 has N and B ($1 \le N \le 10^5$, $0 \le B \le 10^{18}$); line 2 has N integers p[i] ($1 \le p[i] \le 10^9$). **Output:** the maximum count. The running total overflows 32 bits — use `long long`.

```
Input
7 10
3 1 4 1 5 9 2
```

```
Output
4
```

The run 3 1 4 1 costs 9 ≤ 10 and has length 4; no length-5 run fits.

> **Hint.** Prices are positive, so this is the longest/at-most shape. Keep a running `sum` and a left index `l`. Add `p[r]`; while `sum > B`, subtract `p[l]` and advance `l`. The best answer is `r - l + 1` at each step.

8 One-page cheat sheet

	Fixed window	Two pointers
Width	constant K	changes as you sweep
Edges	move together	independent, forward-only
Use when	best block of exactly K	shortest / longest block that satisfies a rule
On enter	`add a[r]`	`add a[r]`
On leave	`subtract a[r-K]`	`subtract a[l]`
Linear because	N slides × O(1)	each pointer travels ≤ n total (amortised)
AIO problem	Mansion (2007)	Wet Chairs (2015)

The two two-pointer shapes	How the pointers move	Example
Longest / at-most	add on the right; while it overshoots, shrink the left	Snack Bar
Shortest / at-least	grow the right until it qualifies, then drop the left	Wet Chairs

If you remember nothing else
If each pointer moves one way only, and every element enters once and leaves once, your sweep is O(N). Find the monotonicity, and the speed follows.

Appendix A — Worked solutions

Try the exercises first. Both solutions below are complete and were checked against a brute force on thousands of random cases and against the provided tests.

Assembly Line — solution

```cpp
#include <iostream>
using namespace std;

const int MAXN = 100005;
long long d[MAXN];

int main() {
    int N, K;
    cin >> N >> K;
    for (int i = 0; i < N; i++) cin >> d[i];

    long long window = 0;
    for (int i = 0; i < K; i++) window += d[i];
    long long best = window;
    for (int i = K; i < N; i++) {
        window += d[i] - d[i - K];
        if (window < best) best = window;
    }
    cout << best << "\n";
    return 0;
}
```

Identical in shape to Mansion: one O(K) build, then an O(1) repair per slide, keeping the minimum. O(N) overall.

Snack Bar — solution

```cpp
#include <iostream>
using namespace std;

const int MAXN = 100005;
long long p[MAXN];

int main() {
    int N;
    long long B;
    cin >> N >> B;
    for (int i = 0; i < N; i++) cin >> p[i];

    long long sum = 0;
    int best = 0;
    int l = 0;
    for (int r = 0; r < N; r++) {
        sum += p[r];
        while (sum > B) {          /* shrink from the left until affordable */
            sum -= p[l];
            l++;
        }
        int len = r - l + 1;       /* window [l, r] is now within budget */
        if (len > best) best = len;
    }
    cout << best << "\n";
    return 0;
}
```

The longest/at-most sweep. Because prices are positive, once `sum ≤ B` holds you never need to move `l` back, so each index advances at most N times: O(N). When a single item exceeds B, the `while` empties the window and the length becomes 0, which is exactly right.

Appendix B — More problems to try

Same idea, more practice. For each, first decide which shape it is, then find the monotonic quantity.

Problem	Year	Nudge
Alien	AIO 2011	Fix one edge of the window; work out the monotone condition on the other.
Cloud Cover	AIO 2018	Maintain a "coverage" or "cost" for the window and update it incrementally as it moves.
Farmer Drama	AIO 2016	You may be sliding over a *transformed* sequence, not the raw array. Ask what quantity becomes monotonic once framed right.

Good luck

Come back to the cheat sheet whenever a new problem smells like "every contiguous block." Nine times out of ten, the answer is a running summary and two forward-only pointers.